

Flink 原理详解 - 第 14 讲

Kafka + Flink 联合实战：从 Topic 到实时窗口、Checkpoint 与 Lag

本讲目标：把 Kafka 和 Flink 真正接起来。学完后，你应该能独立完成“建 Topic -> 写实时数据 -> Flink SQL 消费 -> Watermark/窗口聚合 -> 写回 Kafka -> 查看 Lag -> 验证 Checkpoint”的最小闭环。

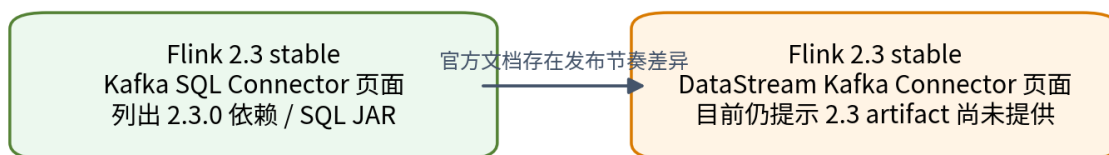
你已经具备的前置知识

- Kafka：知道 Topic、Partition、Producer、Consumer、Consumer Group 的基本概念。
- Flink：已经学习 Event Time、Watermark、Window、State、Checkpoint、Backpressure、Flink SQL 与 Connector。
- Linux：能够在终端执行命令，并能找到 \$KAFKA_HOME 与 \$FLINK_HOME。

本讲为什么选择 Flink SQL 作为实战主线

当前 Flink 2.3 的官方 Kafka SQL Connector 页面已经列出 2.3.0 对应依赖和 SQL Client JAR；与此同时，DataStream Kafka Connector 的稳定文档仍提示 2.3 对应 artifact 尚未提供。为了减少零基础上机时的依赖冲突，本讲采用 SQL Client + Kafka Table Connector。你学到的 Topic、Offset、Watermark、Checkpoint、Lag 等原理完全可以迁移到 DataStream API。

版本现实：本讲为什么用 Flink SQL + Kafka Table Connector



为了让零基础实验更可复现：本讲主线选择 SQL Client + Kafka Table Connector。
概念完全可以迁移到 DataStream API。

图 1 当前版本现实与本讲技术路线

1. 先看全局：Kafka 和 Flink 各自负责什么？

Kafka + Flink 联合实战：端到端数据流

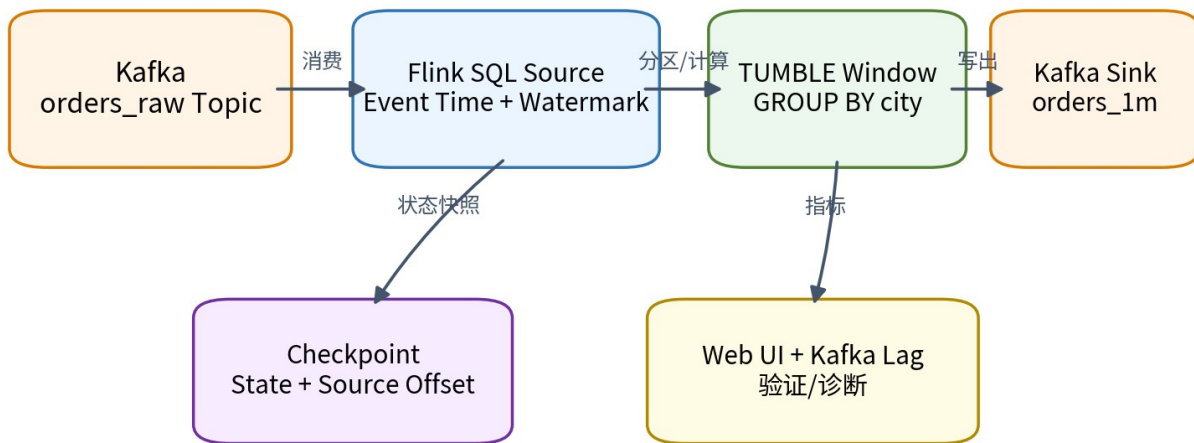


图 2 本讲端到端架构

组件	主要职责	本讲中的对象
Kafka	持续接收、保存、分区化事件流	orders_raw / orders_1m
Flink Source	从 Kafka 持续消费并恢复 Offset	orders_raw 表
Watermark	估计事件时间进度	允许 5 秒乱序
Window	把无界流切成有限计算区间	1 分钟 TUMBLE
Checkpoint	保存 State + Source Position	周期性恢复点
Kafka Sink	把最终窗口结果持续写出	orders_1m 表
Web UI / Lag	判断系统是否健康	Checkpoint、Backpressure、Lag

核心认识：Kafka 负责“可靠保存事件”，Flink 负责“持续计算事件”。Kafka 中的 Offset 是“消费到哪里”，Flink State 是“算到这里时记住了什么”。Checkpoint 要把二者放在同一个一致性恢复点上。

2. 实验前检查：不要一上来就写 SQL

2.1 确认 Kafka 与 Flink 目录

```
echo $KAFKA_HOME
echo $FLINK_HOME
```

如果变量为空，请先进入实际安装目录，或临时 export。

2.2 确认 Kafka 已经运行

能列出 Topic，至少说明 Broker 可连接。

```
$KAFKA_HOME/bin/kafka-topics.sh \  
--bootstrap-server localhost:9092 \  
--list
```

2.3 启动/检查 Flink Standalone Cluster

```
$FLINK_HOME/bin/start-cluster.sh
```

```
# 浏览器打开：  
# http://localhost:8081
```

2.4 检查 Kafka SQL Connector JAR

```
ls $FLINK_HOME/lib | grep -i kafka
```

重要：如果这里没有 Kafka Connector JAR，SQL Client 中 CREATE TABLE 可能报 “Could not find any factory for identifier kafka”。先安装与你 Flink 版本匹配的 Kafka SQL Connector，再继续。不要靠复制别人机器上的任意 JAR 碰运气。

3. 第一步：创建输入和输出 Topic

为了让初学者容易观察，本讲使用单分区 Topic。这样 Watermark、Offset 与 Lag 都更直观。真实生产环境通常会根据吞吐和并行度使用多个 Partition。

```
$KAFKA_HOME/bin/kafka-topics.sh --create \
  --bootstrap-server localhost:9092 \
  --topic orders_raw \
  --partitions 1 \
  --replication-factor 1

$KAFKA_HOME/bin/kafka-topics.sh --create \
  --bootstrap-server localhost:9092 \
  --topic orders_1m \
  --partitions 1 \
  --replication-factor 1
```

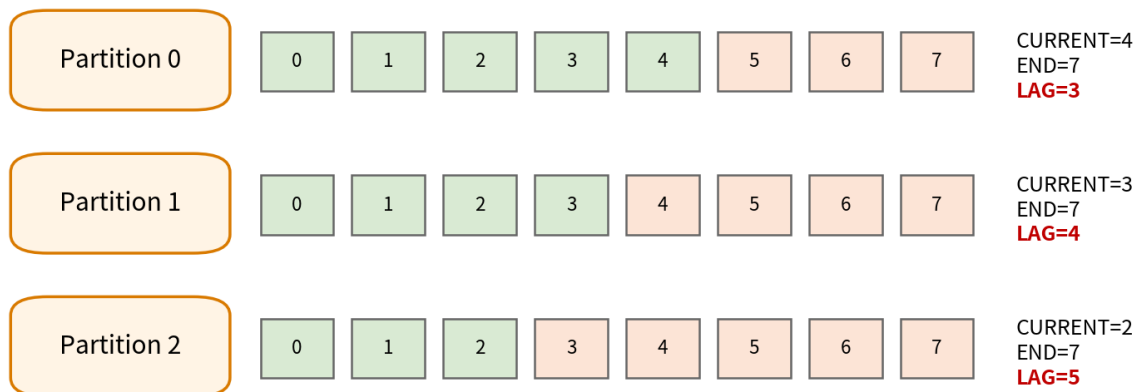
3.1 检查 Topic

```
$KAFKA_HOME/bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --describe --topic orders_raw

$KAFKA_HOME/bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --describe --topic orders_1m
```

3.2 Offset 和 Lag 到底是什么

Kafka Topic / Partition / Offset / Lag



Lag = Log End Offset - Current/Committed Offset
 Lag 增长说明“进入 Kafka 的速度”高于“被消费/提交的速度”。

图 3 Partition、Offset 与 Lag

公式： $Lag = Log\ End\ Offset - Consumer\ Current/Committed\ Offset$ 。Lag 持续增长，说明端到端处理速度跟不上输入速度；但根因可能在 Source、计算、Shuffle、State 或 Sink，不能只怪 Kafka。

4. 第二步：手工写入实时订单 JSON

打开一个新的终端，启动 Kafka Console Producer。每输入一行 JSON，就产生一条 Kafka Event。

```
$KAFKA_HOME/bin/kafka-console-producer.sh \
  --bootstrap-server localhost:9092 \
  --topic orders_raw
```

依次输入下面的数据。为了观察事件时间，我们故意让 event_time 与输入顺序并不完全一致：

```
{ "order_id": "o001", "user_id": "u01", "city": "Beijing", "amount": 100.00, "event_time": "2026-09-29 09:00:05" }
{ "order_id": "o002", "user_id": "u02", "city": "Beijing", "amount": 50.00, "event_time": "2026-09-29 09:00:10" }
{ "order_id": "o003", "user_id": "u03", "city": "Shanghai", "amount": 80.00, "event_time": "2026-09-29 09:00:20" }
{ "order_id": "o004", "user_id": "u04", "city": "Beijing", "amount": 20.00, "event_time": "2026-09-29 09:00:55" }
{ "order_id": "o005", "user_id": "u05", "city": "Shanghai", "amount": 40.00, "event_time": "2026-09-29 09:01:06" }
```

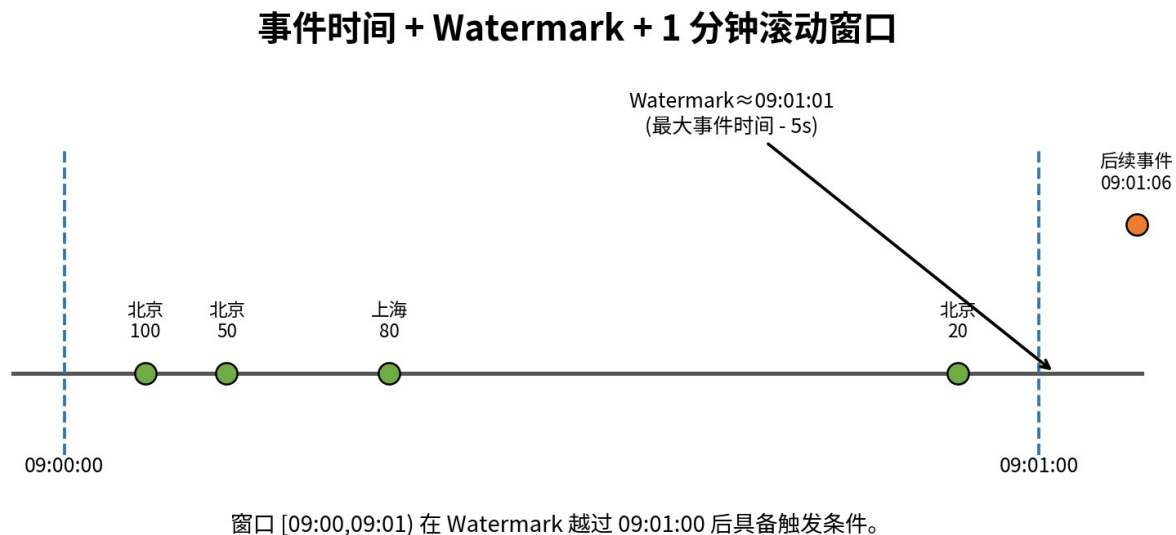


图 4 事件时间、Watermark 与窗口触发

前四条属于 [09:00:00, 09:01:00) 窗口。第五条把“已观察到的最大事件时间”推进到 09:01:06；若 Watermark 策略允许 5 秒乱序，则 Watermark 约推进到 09:01:01，从而越过窗口结束时间 09:01:00，前一个窗口具备触发条件。

5. 第三步：进入 Flink SQL Client

```
$FLINK_HOME/bin/sql-client.sh
```

5.1 先设置实验级 Checkpoint

```
SET 'execution.checkpointing.interval' = '10 s';
SET 'execution.checkpointing.mode' = 'EXACTLY_ONCE';
SET 'execution.checkpointing.storage' = 'filesystem';
SET 'execution.checkpointing.dir' = 'file:///tmp/flink-checkpoints';
```

实验与生产的区别：单机实验可以把 checkpoint 写到本地 file:///tmp/...；生产集群应该写到所有参与节点都能访问的持久化存储，例如 HDFS 或对象存储。否则 TaskManager/机器真正坏掉时，本地快照也可能一起丢失。

5.2 建立 Kafka Source 表

```
CREATE TABLE orders_source (
  order_id STRING,
  user_id STRING,
  city STRING,
  amount DECIMAL(10, 2),
  event_time TIMESTAMP(3),
  WATERMARK FOR event_time AS event_time - INTERVAL '5' SECOND
) WITH (
  'connector' = 'kafka',
  'topic' = 'orders_raw',
  'properties.bootstrap.servers' = 'localhost:9092',
  'properties.group.id' = 'flink-orders-v1',
  'scan.startup.mode' = 'earliest-offset',
  'format' = 'json',
  'json.ignore-parse-errors' = 'true'
);
```

5.3 先不要急着聚合：直接 SELECT 验证 Source

```
SELECT * FROM orders_source;
```

如果能看到刚才输入的 JSON 被正确解析成列，说明 Kafka -> Flink Source 这一段已经打通。此时遇到问题，优先检查 Topic 名、Bootstrap Server、Connector JAR、JSON 字段名和时间格式。

6. 第四步：创建 Kafka Sink 表

```
CREATE TABLE orders_sink (  
    window_start TIMESTAMP(3),  
    window_end TIMESTAMP(3),  
    city STRING,  
    order_count BIGINT,  
    total_amount DECIMAL(20, 2)  
) WITH (  
    'connector' = 'kafka',  
    'topic' = 'orders_1m',  
    'properties.bootstrap.servers' = 'localhost:9092',  
    'format' = 'json',  
    'sink.semantic' = 'at-least-once'  
);
```

为什么先用 at-least-once：第一遍实验先把“数据流正确”跑通。Kafka SQL Connector 在启用 checkpoint 后可以配置 exactly-once sink 语义，但那会引入事务、消费者 isolation.level 等额外概念。先形成最小闭环，再升级语义。

7. 第五步：运行 1 分钟窗口聚合

```
INSERT INTO orders_sink
SELECT
  window_start,
  window_end,
  city,
  COUNT(*) AS order_count,
  CAST(SUM(amount) AS DECIMAL(20, 2)) AS total_amount
FROM TABLE(
  TUMBLE(TABLE orders_source, DESCRIPTOR(event_time), INTERVAL '1' MINUTE)
)
GROUP BY window_start, window_end, city;
```

7.1 这条 SQL 背后发生了什么

1. Kafka Source 持续拉取 orders_raw。
2. JSON 反序列化表中的字段。
3. event_time 参与 Event Time 语义，Watermark = 已观察最大事件时间 - 5 秒。
4. TUMBLE 将事件放入 1 分钟窗口。
5. GROUP BY city 使相同城市的数据在聚合前发生按 Key 的网络重分区（Exchange）。
6. 窗口状态持续累计 COUNT / SUM。
7. Watermark 越过窗口结束时间后，窗口结果被产生。
8. 结果序列化为 JSON 写入 orders_1m。

7.2 手工验算第一个窗口

城市	订单	数量	总金额
Beijing	o001=100, o002=50, o004=20	3	170.00
Shanghai	o003=80	1	80.00

验收标准：如果输出不是这两个结果，先不要讨论性能。第一优先级是确认业务正确性：字段解析、Event Time、窗口边界、金额类型和输入数据是否符合预期。

8. 第六步：从 Kafka 读取 Flink 的窗口结果

```
$KAFKA_HOME/bin/kafka-console-consumer.sh \
--bootstrap-server localhost:9092 \
--topic orders_1m \
--from-beginning
```

你应该看到类似 JSON 结果。输出顺序不应该被当成业务语义的一部分；Kafka 多分区后，全局顺序更不能假定。

9. 第七步：看 Consumer Group 与 Lag

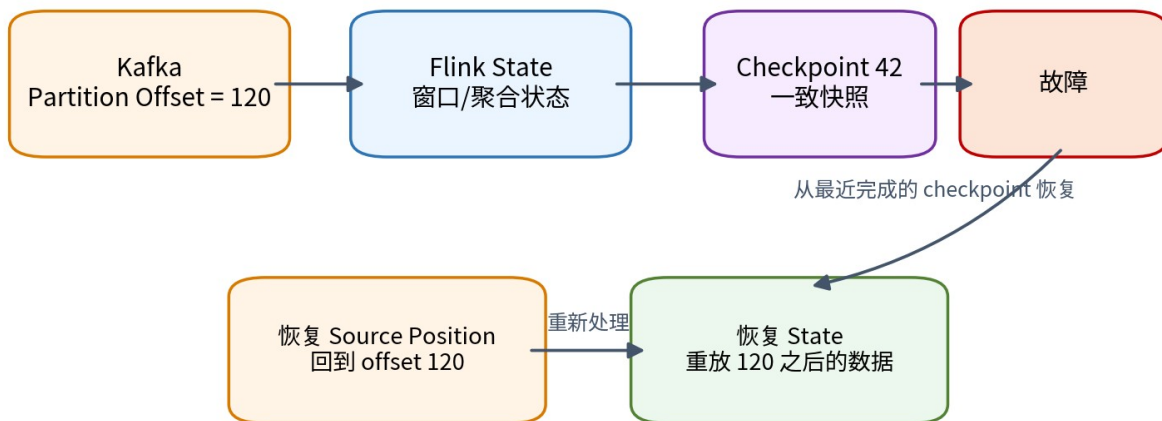
```
$KAFKA_HOME/bin/kafka-consumer-groups.sh \
--bootstrap-server localhost:9092 \
--describe \
--group flink-orders-v1
```

重点看 CURRENT-OFFSET、LOG-END-OFFSET 与 LAG。如果持续往 orders_raw 写数据而 Flink 停止/变慢，LAG 会增加。

关键诊断原则：Kafka Lag 是“症状”，不是根因。Source 显示 Lag 大时，仍要沿 Flink Web UI 向下游检查：哪个算子 busy、哪个算子 backpressured、Checkpoint 是否变慢、Sink 是否吞吐不足。

10. 第八步：把 Checkpoint 与 Kafka Offset 联系起来

Checkpoint：为什么 Kafka 能和 Flink 一起恢复



Checkpoint 需要同时保存“处理到哪里”和“当时记住了什么”。

图 5 Checkpoint 将 Source Position 与 State 放在同一恢复点

Kafka 是可重放的数据源。Flink 的 Checkpoint 不需要把 Kafka 中的所有消息复制一份，而是保存“这个一致快照对应消费到了哪些 Offset”，同时保存窗口、聚合等 State。故障后恢复 State，并从对应 Source Position 继续或重放。

10.1 检查本地 Checkpoint 文件

```
find /tmp/flink-checkpoints -maxdepth 3 -type f | head -30
```

10.2 在 Web UI 中观察

- 打开 <http://localhost:8081>，进入当前 Job。
- 打开 Checkpoints 页面：确认 Completed Checkpoint 在持续产生。
- 记录 End-to-End Duration、Checkpoint Size、Alignment/Start Delay。
- 回到 Job Graph：观察 Source、Window/Aggregate、Sink 各算子的 Busy / Backpressure。

10.3 一个严谨的恢复实验应该验证什么

检查项	故障前	恢复后应该看到什么
Source Position	Kafka 已消费到某 Offset	从最近完成 Checkpoint 对应位置恢复/重放
Window State	已有未关闭窗口口的 COUNT/SUM	未关闭窗口状态继续存在
最终结果	已关闭窗口已输出	不应因为一次恢复产生错误业务总和
Checkpoint	持续 Completed	恢复后重新继续产生

11. 可选进阶：升级 Kafka Sink 到 Exactly-once

当前 Kafka SQL Connector 文档说明：启用 Flink Checkpoint 后，可以通过 `sink.semantic = exactly-once` 使用 Kafka 事务提供精确一次 Sink 语义。进行这一实验前，要先理解“Flink State Exactly-once”和“端到端 Sink Exactly-once”不是同一个层次。

```
-- 在版本/Connector 明确支持时，可将 Sink 选项改为：
'sink.semantic' = 'exactly-once'

-- 下游消费者若只希望读取已提交事务的数据，
-- 还要正确配置 Kafka consumer isolation.level。
```

不要机械追求 exactly-once：Exactly-once 往往会增加事务协调和延迟成本。业务是否需要它，应由“重复结果能否接受、能否幂等、恢复语义要求”决定，而不是看到选项就全部打开。

12. 三类最常见故障：按层排查

现象	优先检查	常见原因
CREATE TABLE 报 kafka factory 不存在	Connector/JAR	Kafka SQL Connector 未安装或版本不匹配
SELECT 没数据	Kafka/Source	Topic 名错、startup mode、group offset、Broker 地址
JSON 解析 NULL/缺字段	Schema/Format	字段名/类型/时间格式不一致
窗口迟迟不输出	Time/Watermark	没有新事件推进 Watermark、分区空闲阻塞、事件时间错误
Kafka Lag 持续增大	全链路	下游慢、Backpressure、热点 Key、Sink 慢
Checkpoint 持续失败	State/Storage	目录不可写、存储不可达、超时、反压严重
结果重复	语义/Sink	at-least-once 恢复重放、Sink 非幂等

12.1 典型排错顺序

1. Kafka Broker 是否可连？Topic 是否存在？
2. Kafka 中是否真的有数据？先用 console-consumer 验证。
3. Flink Connector JAR 是否存在？
4. Source 表 SELECT 是否能正常读到并解析？
5. Event Time / Watermark 是否在推进？
6. Window/Aggregate 是否产生结果？
7. Sink 是否能写？
8. 最后再看性能、Lag、Checkpoint 与 Backpressure。

13. 进阶实验：制造“窗口不触发”

这是非常值得亲手做一次的实验：只写入 09:00:05、09:00:10、09:00:20 三条事件，然后停止生产。你可能发现 [09:00,09:01) 窗口并不会立刻输出，因为 Watermark 没有足够的新事件推动它越过 09:01:00。

底层原因：Watermark 不是墙上时钟。它是基于事件流进度生成的时间进度信号。如果事件源彻底没有新数据，Event Time 可能不会自动像 Processing Time 那样持续向前。

再输入一条 09:01:06 的事件

```
{"order_id":"o006","user_id":"u06","city":"Beijing","amount":1.00,"event_time":"2026-09-29 09:01:06"}
```

此时最大事件时间前进，Watermark 随之推进，前一个 1 分钟窗口具备触发条件。这就是第 3 讲 Watermark 原理在真实 Kafka 流里的具体表现。

14. 多 Partition 以后为什么更复杂

Kafka Connector 可以按 Partition 生成 Watermark，而 Source 输出 Watermark 需要考虑多个输入分区。一个长时间没有数据的 Partition 可能拖住整体 Event Time 进度。生产系统因此需要关注 idle partition，并根据业务设置 source idle timeout。

单分区实验	多分区生产环境
Watermark 容易推演	多个 Partition 各自推进
Lag 只有一行	每个 Partition 都有 Offset/Lag
并行度低	Source Parallelism 可提升吞吐
没有分区热点	Key/Partition 不均衡可能形成热点
更适合教学	更接近真实系统

15. 一次完整闭环应该留下哪些“证据”？

本讲上机流程

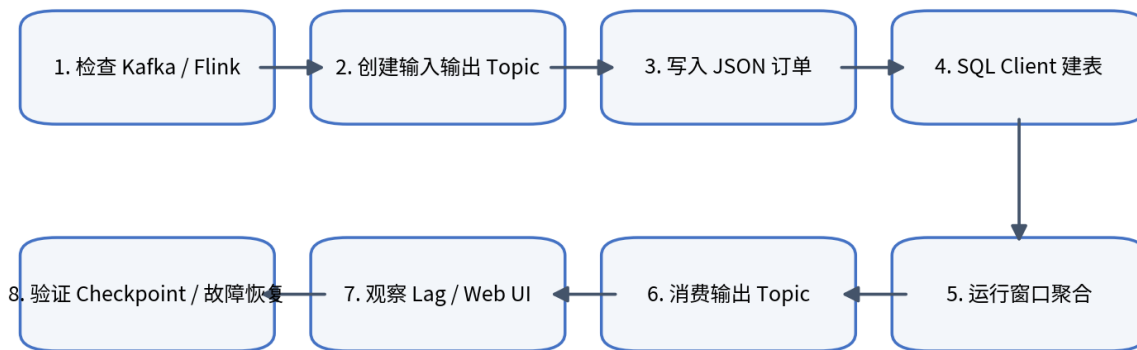


图 6 联合实战的最小闭环

证据	你要确认什么
Kafka Topic	输入/输出 Topic 存在，Partition 数符合预期
原始消息	console-consumer 能看到 orders_raw JSON
Flink Source	SELECT * 能正确解析
业务结果	窗口 COUNT/SUM 与手算答案一致
Kafka Sink	orders_1m 能读取到结果
Consumer Group	能看到 offset 和 lag
Web UI	Job Running，算子指标合理
Checkpoint	持续 Completed，目录有持久化数据

最低闭环能力：不要把“命令执行成功”当成实验完成。一个合格的分布式流处理实验至少需要：输入证据 + 执行证据 + 状态/容错证据 + 业务结果证据。

16. 常见误区

误区	正确理解
Kafka 有数据，Flink 就一定会出窗口结果	还需要正确 Event Time、Watermark 和窗口触发条件
Kafka Lag 大就是 Source 太慢	Lag 可能由任意下游瓶颈反向传播造成
Checkpoint 就是提交 Kafka Offset	Checkpoint 同时保护 Flink State 与 Source Position
EXACTLY_ONCE 表示代码物理只执行一次	故障时可能重放；保证的是一致语义
窗口输出顺序应该固定	分布式/多分区下不要依赖全局输出顺序
Connector JAR 随便找个版本就能用	Connector 与 Flink 版本必须兼容
实验用本地 checkpoint 和生产一样安全	单机 file:// 不能替代真正共享持久化存储

17. 练习题

1. 把 1 分钟滚动窗口改成 2 分钟，你需要改哪一处 SQL？
2. 如果 Watermark 是 `event_time - 5 秒`，那么最大事件时间为 10:01:10 时，Watermark 大约是多少？
3. Kafka Lag 从 0 持续增长到 100000，你第一步应该只调 Source 并行度吗？为什么？
4. 为什么 Checkpoint 恢复必须同时恢复 Source Position 和 Window State？
5. 如果 Sink 是 at-least-once，故障恢复后为什么可能看到重复输出？
6. 多 Partition 时，某个 Partition 一直没有数据，为什么可能影响 Watermark？

参考答案

1. 把 TUMBLE 的 INTERVAL '1' MINUTE 改成 INTERVAL '2' MINUTE。
2. 约为 10:01:05。
3. 不能。先从 Web UI 判断真正的瓶颈位置；下游慢也会造成 Source 反压和 Kafka Lag。
4. 只恢复 Offset 会丢失已经累计的计算状态；只恢复 State 又不知道应从 Kafka 哪里继续，两者必须一致。
5. 恢复时可能从旧 Offset 重放一段数据，at-least-once Sink 允许重复写出。
6. Source 的整体 Watermark 需要综合各 Partition 的进度；空闲分区如果未被识别为 idle，可能拖住整体时间进度。

18. 本讲总结：把 Kafka + Flink 压缩成 10 句话

1. Kafka 把持续事件保存成按 Partition 排列的日志。
2. Offset 表示某个 Partition 中的位置，Lag 表示消费进度落后多少。
3. Flink Source 把 Kafka 事件变成持续 DataStream / Dynamic Table。

4. Event Time 描述业务事件真正发生的时间。
5. Watermark 描述系统对 Event Time 进度的估计。
6. Window 把无界流切成可触发的有限统计区间。
7. State 记住尚未完成的窗口与聚合中间结果。
8. Checkpoint 把 State 与 Source Position 放在同一个一致恢复点。
9. Backpressure 会从下游反向传播，并最终表现成 Kafka Lag 增长。
10. 真正的完成不是“Job Running”，而是输入、状态、恢复、输出和业务结果都能被验证。

Kafka + Flink 联合实战：端到端数据流

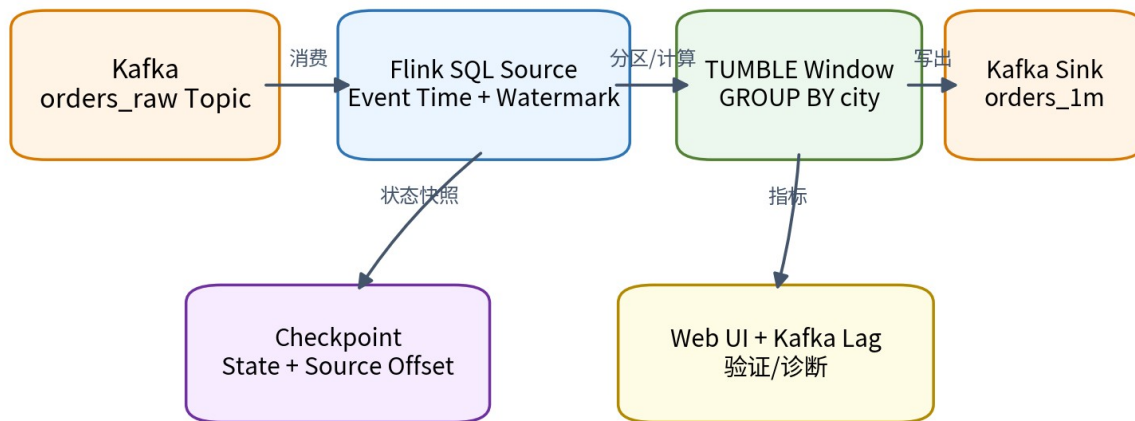


图 7 回看完整端到端数据流

官方资料核对（截至 2026-09-29）

- Apache Flink Stable Documentation - Kafka SQL Connector: Kafka Source/Sink、startup mode、sink.semantic、partition watermark。
- Apache Flink Stable Documentation - Checkpointing: Checkpoint 保存状态与流位置，用于故障恢复。
- Apache Kafka Quickstart / Operations: Topic、console producer/consumer、consumer group 与 lag。